

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

Факультет информационных технологий
Кафедра «Инфокогнитивные технологии»

КУРСОВОЙ ПРОЕКТ

по дисциплине «Разработка мобильных приложений»

на тему: «Разработка приложения «Тренажёр дробей» для изучения дробей»

Направление подготовки 09.03.03 «Прикладная информатика»
Профиль «Корпоративные информационные системы»

Выполнил:
студент группы 231-361
Кузнецов Владислав Владимирович

Москва 2026

Введение

Дроби являются одной из фундаментальных тем школьной программы по математике. Многим учащимся трудно усваивать этот материал, поскольку он требует одновременного понимания числителя, знаменателя, эквивалентности и сравнения дробей. Классические учебники ограничиваются статичными иллюстрациями, а упражнения сводятся к вычислению результатов на бумаге. При таком подходе учащийся не получает возможности наглядно представить дробь как часть целого и экспериментировать с различными формами.

В рамках дисциплины «Разработка мобильных приложений» разработано приложение FractionTrainer — тренажёр для изучения дробей. Приложение показывает геометрическую фигуру (круг, прямоугольник, треугольник, пятиугольник, шестиугольник, ромб, крест, звезду), разделённую на N равных секторов, и предлагает пользователю выбрать нужное количество частей для получения заданной дроби. Все элементы интерфейса выполнены на русском языке.

Практическая ценность проекта определяется тем, что он объединяет два режима работы: обучение и проверку знаний. В режиме обучения пользователь может создавать дроби без ограничений по времени и получать подсказки. В режиме проверки ответы оцениваются, а результаты отображаются в виде счётчика правильных ответов. Предусмотрены три уровня сложности, каждому из которых соответствует набор допустимых знаменателей.

Помимо основного режима сбора дробей, приложение поддерживает пять дополнительных типов заданий: определение дроби по визуальному представлению, сравнение двух дробей, дополнение дроби до целого, поиск эквивалентной дроби из четырёх вариантов и карточная игра на поиск пар эквивалентных дробей. Все задания генерируются случайным образом с учётом выбранного уровня сложности.

Программный код разделён на четыре проекта: библиотеку основной логики, WPF-приложение, консольную версию и набор тестов. Такое разделение обеспечивает независимость бизнес-логики от пользовательского

интерфейса и позволяет повторно использовать модель данных в других типах приложений.

1 Цель и задачи работы

Целью работы является разработка приложения FractionTrainer, обеспечивающего интерактивное изучение дробей с использованием визуальных геометрических фигур и шести различных типов заданий.

Для достижения поставленной цели необходимо решить следующие задачи:

1. определить функциональные требования и основные пользовательские сценарии;
2. спроектировать структуру приложения с разделением логики и интерфейса;
3. разработать модель дроби с поддержкой сокращения и проверки эквивалентности;
4. разработать модель головоломки с управлением набором секторов;
5. реализовать генератор заданий для шести типов задач;
6. реализовать кастомный WPF-компонент для отрисовки восьми типов геометрических фигур;
7. реализовать два режима работы: обучение и проверку;
8. реализовать три уровня сложности с набором допустимых знаменателей;
9. реализовать интерфейс в архитектуре MVVM с привязками данных и командами;
10. реализовать хранение настроек и результатов в формате JSON;
11. реализовать поддержку светлой и тёмной тем оформления;
12. реализовать консольную версию приложения;
13. разработать набор автоматизированных тестов;
14. подготовить приложение к демонстрации и дальнейшему расширению.

В качестве языка программирования используется C#, а графический интерфейс реализован средствами Windows Presentation Foundation. Проект ориентирован на операционную систему Windows и использует возможности платформы .NET. Для хранения настроек и результатов применяется формат JSON.

2 Проектирование приложения

2.1 Функциональные требования

К основным функциональным требованиям относятся создание головоломки с заданным уровнем сложности, отрисовка геометрической фигуры с секторами, управление набором выбранных секторов, проверка правильности ответа и отображение обратной связи.

Пользователь должен иметь возможность выполнять следующие действия:

- выбирать уровень сложности: лёгкий, средний или сложный;
- выбирать режим работы: обучение или проверка;
- генерировать новую задачу случайного типа;
- выбирать секторы фигуры кликом мыши;
- вводить числитель и знаменатель вручную;
- выбирать правильный ответ из предложенных вариантов;
- находить пары эквивалентных дробей на карточной доске;
- проверять свой ответ и получать обратную связь;
- просматривать правильный ответ;
- сбрасывать текущую задачу;
- менять тему оформления и акцентный цвет;
- сохранять и восстанавливать пользовательские настройки;
- использовать сочетания клавиш для быстрого управления.

2.2 Архитектура приложения

Приложение построено по модульному принципу и состоит из четырёх проектов. Проект `FractionTrainer.Core` содержит

модели данных, перечисления, интерфейсы сервисов, генератор заданий и сервисы хранения. Проект `FractionTrainer.WPF` содержит графическое приложение, реализованное по паттерну MVVM. Проект `FractionTrainer.Console` предоставляет консольную версию тренажёра. Проект `FractionTrainer.Tests` содержит модульные тесты на базе xUnit.

На рисунке 1 представлена общая архитектура приложения.

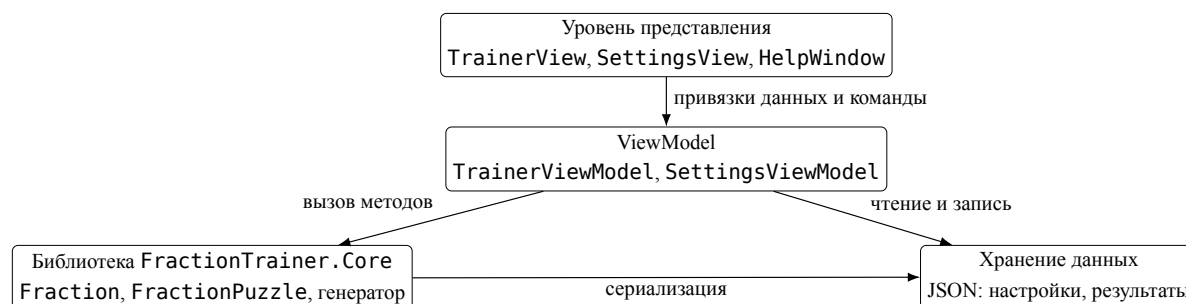


Рисунок 1 – Архитектура приложения FractionTrainer

Такое разделение уменьшает связанность компонентов. Библиотека основной логики не зависит от элементов управления WPF и может использоваться в консольном приложении или в другом типе интерфейса. ViewModel управляет состоянием интерфейса через команды и привязки данных, не обращаясь напрямую к элементам управления.

2.3 Структура классов

Центральной сущностью основной логики является класс `Fraction`. Он хранит целочисленные числитель и знаменатель, предоставляет методы сокращения, проверки эквивалентности и преобразования в вещественное число. Класс `FractionPuzzle` управляет набором выбранных секторов и определяет решение головоломки. Модели `ComparePuzzle`, `CompleteToWholePuzzle`, `EquivalentPuzzle` и `FindPairsPuzzle` реализуют специфическую логику для каждого типа задания.

На рисунке 2 показаны основные классы приложения и связи между ними.

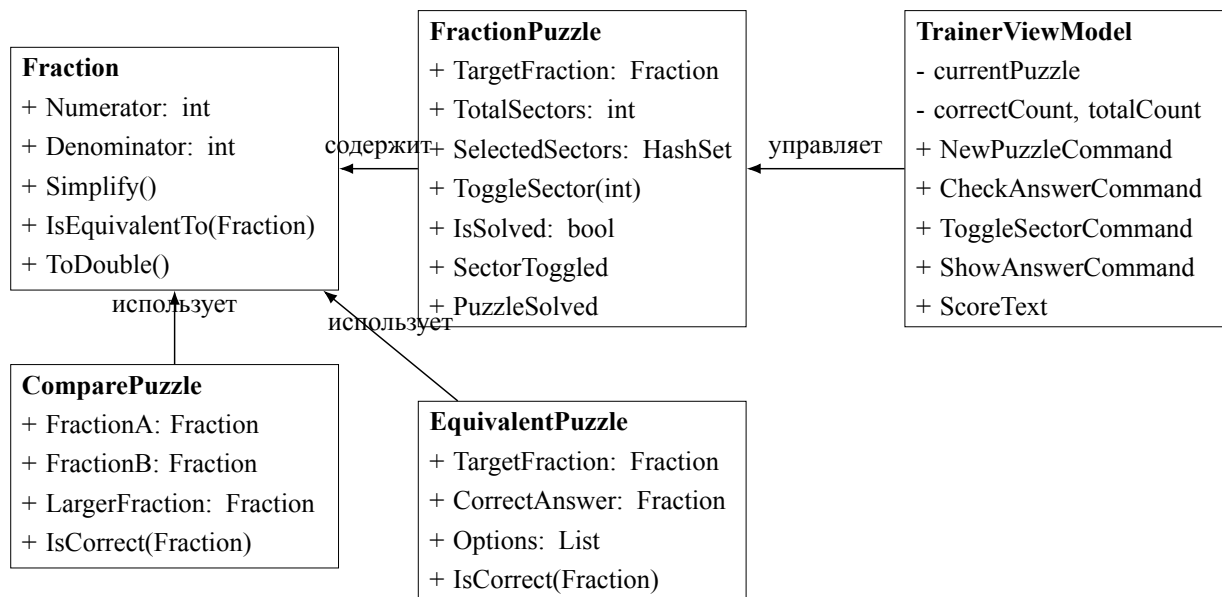


Рисунок 2 – Диаграмма основных классов приложения

2.4 Варианты использования

Основным актором системы является пользователь. После запуска приложения ему доступны все функции тренажёра. Авторизация и серверная часть не требуются, поскольку проект является локальным графическим приложением.



Рисунок 3 – Диаграмма вариантов использования

2.5 Уровни сложности и генерация заданий

Приложение поддерживает три уровня сложности. Каждому уровню соответствует набор допустимых знаменателей для генерации случайных дробей.

Table 1: Знаменатели по уровням сложности

Уровень	Допустимые знаменатели
Лёгкий (Easy)	2, 3, 4
Средний (Medium)	2, 3, 4, 5, 6
Сложный (Hard)	2, 3, 4, 5, 6, 8, 10

Генератор случайных заданий выбирает знаменатель из соответствующего массива и случайный числитель от 1 до знаменателя минус 1, чтобы дробь была правильной. Для заданий на сравнение генерируются две дроби с гарантией их неэквивалентности. Для поиска эквивалентных дробей используется алгоритм умножения числителя и знаменателя на один и тот же множитель с проверкой результата на вхождение в допустимый диапазон.

2.6 Структура настроек приложения

Настройки приложения хранятся в JSON-файле в каталоге %AppData%/FractionTrainer/settings.json. Настройки включают тему оформления, акцентный цвет, уровень сложности по умолчанию и режим работы.

Table 2: Параметры настроек приложения

Поле	Назначение
Theme	Тема оформления: «Light» или «Dark»
AccentColor	Акцентный цвет в формате HEX
DefaultDifficulty	Уровень сложности по умолчанию
DefaultMode	Режим работы: обучение или проверка
ShowHints	Отображать ли подсказки

2.7 Проектирование интерфейса

Интерфейс графического приложения разделён на две основные области. Слева располагается боковая панель навигации с тремя кнопками: «Тренажёр», «Настройки» и «Справка». Справа находится контентная область, в которой переключаются соответствующие вкладки.

На вкладке тренажёра размещается заголовок с типом задания, геометрическая фигура с секторами, элементы ввода ответа и кнопки управления. На вкладке настроек — элементы выбора темы, акцентного цвета, уровня сложности и режима работы. На вкладке справки — описание горячих клавиш и основных действий.

В приложении используются две темы: светлая и тёмная. Каждая тема определяет десять кистей с одинаковыми ключами. Все элементы интерфейса привязаны к ним через `DynamicResource`, что обеспечивает автоматическое обновление при переключении темы.

3 Реализация приложения

3.1 Средства разработки

Проект реализован на языке C# для платформы .NET. Графический интерфейс описан на языке XAML, а обработка событий и управление состоянием выполняются в коде C#. Модель представления данных реализована по паттерну MVVM: View (XAML) отображает данные, ViewModel управляет логикой, Model хранит состояние.

Решение состоит из четырёх проектов:

- `FractionTrainer.Core` — библиотека классов с моделями, перечислениями, интерфейсами и сервисами;
- `FractionTrainer.WPF` — WPF-приложение с пользовательским интерфейсом;
- `FractionTrainer.Console` — консольная версия тренажёра;
- `FractionTrainer.Tests` — модульные тесты на базе xUnit.

3.2 Модель дроби

Класс `Fraction` является неизменяемым (`immutable`) и хранит целочисленные числитель и знаменатель. В конструкторе выполняется

проверка, что знаменатель не равен нулю. Метод `Simplify()` сокращает дробь с помощью алгоритма Евклида для нахождения наибольшего общего делителя. Метод `IsEquivalentTo` проверяет эквивалентность двух дробей перекрёстным умножением. Фрагмент модели приведён в листинге 1.

```
1 public class Fraction : IEquatable<Fraction>
2 {
3     public int Numerator { get; }
4     public int Denominator { get; }
5
6     public Fraction(int numerator, int denominator)
7     {
8         if (denominator == 0)
9             throw new ArgumentException(
10                 "Знаменатель не может быть равен нулю.",
11                 nameof(denominator));
12
13         Numerator = numerator;
14         Denominator = denominator;
15     }
16
17     public bool IsEquivalentTo(Fraction other)
18     {
19         return Numerator * other.Denominator
20             == Denominator * other.Numerator;
21     }
22
23     public Fraction Simplify()
24     {
25         int gcd = CalculateGreatestCommonDivisor(
26             Math.Abs(Numerator),
27             Math.Abs(Denominator));
28         return new Fraction(
29             Numerator / gcd,
30             Denominator / gcd);
31     }
32
33     private static int CalculateGreatestCommonDivisor(
34         int a, int b)
35     {
36         while (b != 0)
37         {
```

```

38         int temp = b;
39         b = a % b;
40         a = temp;
41     }
42     return a;
43 }
44 }

```

Листинг 1 – Модель дроби с сокращением и проверкой эквивалентности

3.3 Модель головоломки

Класс `FractionPuzzle` хранит целевую дробь, общее число секторов и множество `HashSet<int>` — индексы выбранных пользователем секторов. Метод `ToggleSector` добавляет или убирает индекс из множества. Свойство `IsSolved` возвращает `true`, когда количество выбранных секторов совпадает с числителем, а общее число секторов — со знаменателем. При решении головоломки срабатывает событие `PuzzleSolved`.

Для каждого типа задания предусмотрен отдельный класс модели. `ComparePuzzle` хранит две дроби и вычисляет, какая из них больше. `CompleteToWholePuzzle` вычисляет недостающую дробь как разность между целым и заданной дробью. `EquivalentPuzzle` хранит целевую дробь, правильный ответ и список из четырёх вариантов. `FindPairsPuzzle` описывает карточную игру с шестью карточками, на которых изображены дроби various форм.

3.4 Генератор заданий

Класс `PuzzleGenerator` реализует интерфейс `IPuzzleGenerator` и создаёт случайные головоломки. Метод `GenerateWithType` создаёт базовую головоломку с указанного типа и случайной геометрической фигурой. Метод `GenerateCompare` генерирует две неэквивалентные дроби с помощью цикла `do-while`. Метод `GenerateEquivalent` формирует список из четырёх вариантов, где только один эквивалентен целевой дроби. Метод `GenerateFindPairs` создаёт карточную доску с шестью карточками, содержащими одну пару эквивалентных дробей и четыре «отвлечения». Генерация заданий показана

В листинге 2.

```
1 public ComparePuzzle GenerateCompare(DifficultyLevel difficulty)
2 {
3     Fraction a = GenerateRandomFraction(difficulty);
4     Fraction b;
5     do
6     {
7         b = GenerateRandomFraction(difficulty);
8     }
9     while (a.IsEquivalentTo(b));
10    return new ComparePuzzle(a, b);
11 }
12
13 public EquivalentPuzzle GenerateEquivalent(
14     DifficultyLevel difficulty)
15 {
16     int[] denominators = GetDenominators(difficulty);
17     int targetDenom = denominators[
18         _random.Next(denominators.Length)];
19     int targetNum = _random.Next(1, targetDenom);
20
21     Fraction target = new Fraction(targetNum, targetDenom);
22     Fraction simplified = target.Simplify();
23
24     List<Fraction> options = new List<Fraction>();
25     options.Add(simplified);
26
27     while (options.Count < 4)
28     {
29         int optDenom = denominators[
30             _random.Next(denominators.Length)];
31         int optNum = _random.Next(1, optDenom);
32         Fraction opt = new Fraction(optNum, optDenom);
33         if (!options.Any(o => o.IsEquivalentTo(opt))
34             && !opt.IsEquivalentTo(target))
35         {
36             options.Add(opt);
37         }
38     }
39
40     options = options.OrderBy(_ => _random.Next())
```

```

41         .ToList();
42     return new EquivalentPuzzle(target, simplified, options);
43 }

```

Листинг 2 – Генерация заданий для разных типов задач

3.5 Кастомный компонент **FractionCircleControl**

Класс **FractionCircleControl** является пользовательским WPF-компонентом (**UserControl**) и отрисовывает восемь типов геометрических фигур программно с помощью **PathGeometry**. Поддерживаются следующие типы фигур: круг, прямоугольник, треугольник, пятиугольник, шестиугольник, ромб, крест и звезда.

Компонент определяет пять зависимых свойств (**DependencyProperty**): **TotalSectors**, **SelectedSectors**, **DefaultSectorColor**, **SelectedSectorColor** и **Shape**. При изменении любого из них вызывается метод **DrawShape**, который перерисовывает фигуру.

Для круга вычисляется центр, радиус и угол каждого сектора. Каждый сектор рисуется как **PathFigure**: линия от центра до края, дуга вдоль окружности, линия обратно к центру. Для прямоугольника ширина делится на равные части. Для треугольника используется интерполяция точек по краям. Для пятиугольника и шестиугольника рисуется контур многоугольника, а внутри — секторы от центра. Для звезды используются внешний и внутренний радиусы с чередующимися вершинами.

Каждый сектор — это **Path** с заливкой, зависящей от выделения, и тегом с индексом. При клике по сектору вызывается кастомное маршрутное событие **SectorClicked**, которое всплывает вверх по дереву элементов.

3.6 **ViewModel** тренажёра

Класс **TrainerViewModel** наследует **ObservableObject** и реализует **INotifyPropertyChanged**. Он хранит всё состояние приложения: текущую головоломку, выбранные секторы, текст обратной связи, счётчик правильных ответов и типы ввода для различных заданий. Определены шесть команд: **NewPuzzleCommand**, **ResetCommand**, **CheckAnswerCommand**, **ShowAnswerCommand**,

ToggleSectorCommand, SelectPairCardCommand.

При нажатии «Новая задача» вызывается метод `ExecuteNewPuzzle`, который случайно выбирает один из шести типов заданий и вызывает соответствующий метод генерации. Ключевой фрагмент показан в листинге 3.

```
1 public void ExecuteNewPuzzle()
2 {
3     _currentPuzzleType = (PuzzleType)
4         new Random().Next(0, 6);
5
6     ResetInputState();
7
8     switch (_currentPuzzleType)
9     {
10         case PuzzleType.BuildFraction:
11             GenerateBuildFraction();
12             break;
13         case PuzzleType.IdentifyFraction:
14             GenerateIdentifyFraction();
15             break;
16         case PuzzleType.CompareFractions:
17             GenerateCompareFractions();
18             break;
19         case PuzzleType.CompleteToWhole:
20             GenerateCompleteToWhole();
21             break;
22         case PuzzleType.FindEquivalent:
23             GenerateFindEquivalent();
24             break;
25         case PuzzleType.FindPairs:
26             GenerateFindPairs();
27             break;
28     }
29 }
```

Листинг 3 – Выбор типа задания в ViewModel

Проверка ответа выполняется методом `ExecuteCheckAnswer`. Для каждого типа задания своя логика проверки: для сбора дроби проверяется свойство `IsSolved`, для определения дроби создаётся объект `Fraction` из введённых данных и проверяется эквивалентность, для сравнения

проверяется выбранный индекс. После проверки показывается обратная связь: зелёный текст «Правильно!» или красный с правильным ответом. Таймер `DispatcherTimer` через 1,5 секунды очищает обратную связь.

Для поиска пар реализована отдельная логика: пользователь выбирает первую карточку, затем вторую. Если дроби эквивалентны — обе помечаются как найденные с зелёной рамкой. Если нет — показывается ошибка и сбрасывается выбор. Когда все пары найдены, отображается сообщение «Все пары найдены!».

3.7 Темы оформления

Два файла — `LightTheme.xaml` и `DarkTheme.xaml` — определяют десять кистей с одинаковыми ключами: `AppBackground`, `AppForeground`, `SidebarBackground`, `AccentColor` и другие. Все элементы интерфейса привязаны к ним через `DynamicResource`. Переключение темы происходит в методе `App.SwitchTheme`: удаляется старый `ResourceDictionary` из `MergedDictionaries`, добавляется новый. Всё перерисовывается автоматически благодаря привязкам.

3.8 Хранение настроек и результатов

Два сервиса отвечают за целостность данных. `SettingsService` сохраняет и загружает настройки (тема, цвет, уровень сложности, режим) в файл `%AppData%/FractionTrainer/settings.json`. `JsonScoreRepository` сохраняет результаты сессий в файл `scores.json`. Оба используют `System.Text.Json` для сериализации и реализуют интерфейсы — `ISettingsService` и `IScoreRepository` — что позволяет заменить способ хранения без изменения остального кода.

3.9 MVVM-инфраструктура

Два файла в папке `Infrastructure` обеспечивают работу паттерна MVVM. Класс `ObservableObject` реализует `INotifyPropertyChanged` с методом `OnPropertyChanged`, использующий атрибут `[CallerMemberName]` для автоматического определения имени свойства. Класс `RelayCommand` реализует `ICommand`, принимая делегат `Action` для выполнения и `Func<bool>` для проверки доступности. Команды привязаны

к `CommandManager.RequerySuggested` для автоматического обновления состояния кнопок.

3.10 Алгоритм обработки клика по сектору

Последовательность обработки щелчка по сектору фигуры представлена на рисунке 4.

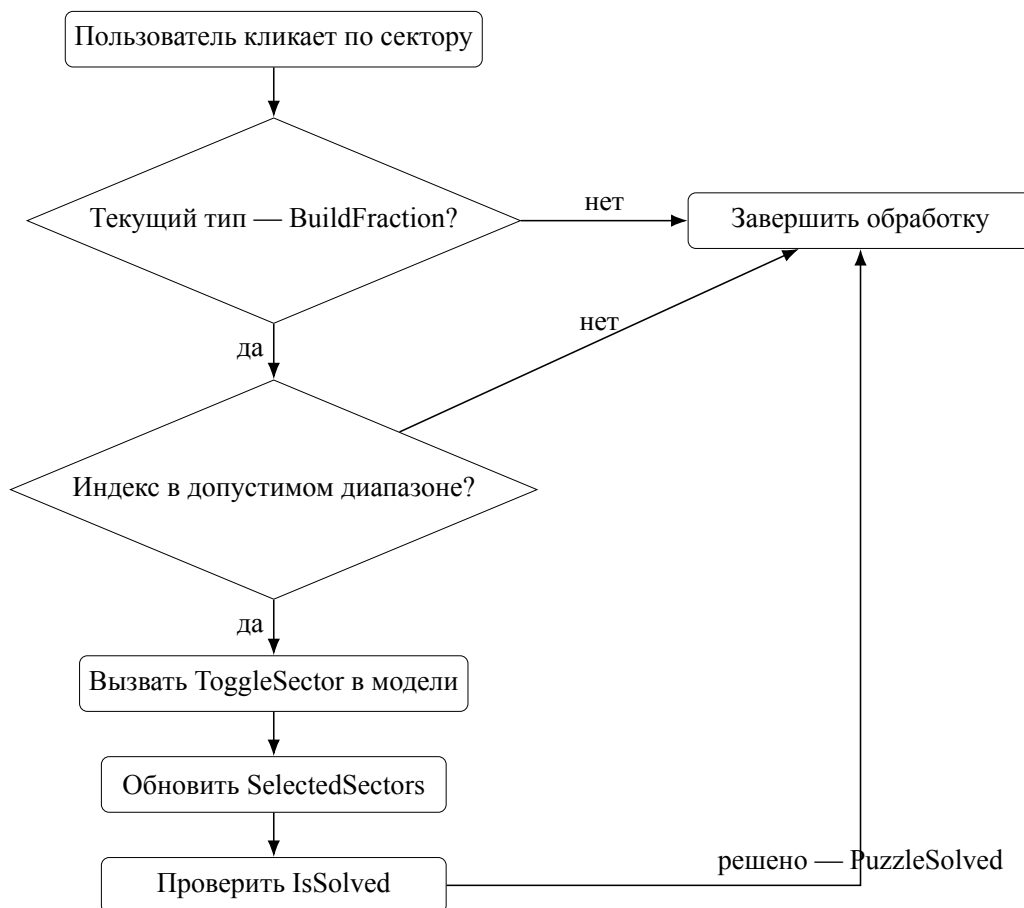


Рисунок 4 – Алгоритм обработки клика по сектору

4 Основные сценарии использования приложения

4.1 Сбор дроби из секторов

После запуска приложения пользователь попадает на вкладку тренажёра. Случайным образом выбирается тип задания. Если выбран режим «Собрать дробь», на экране отображается геометрическая фигура с набором секторов. Пользователь кликает по секторам, чтобы выбрать нужное количество частей. Текущая дробь обновляется в реальном времени. На рисунке 5 показан режим сбора дроби: фигура прямоугольника разделена на четыре части, пользователь выбрал одну из них.

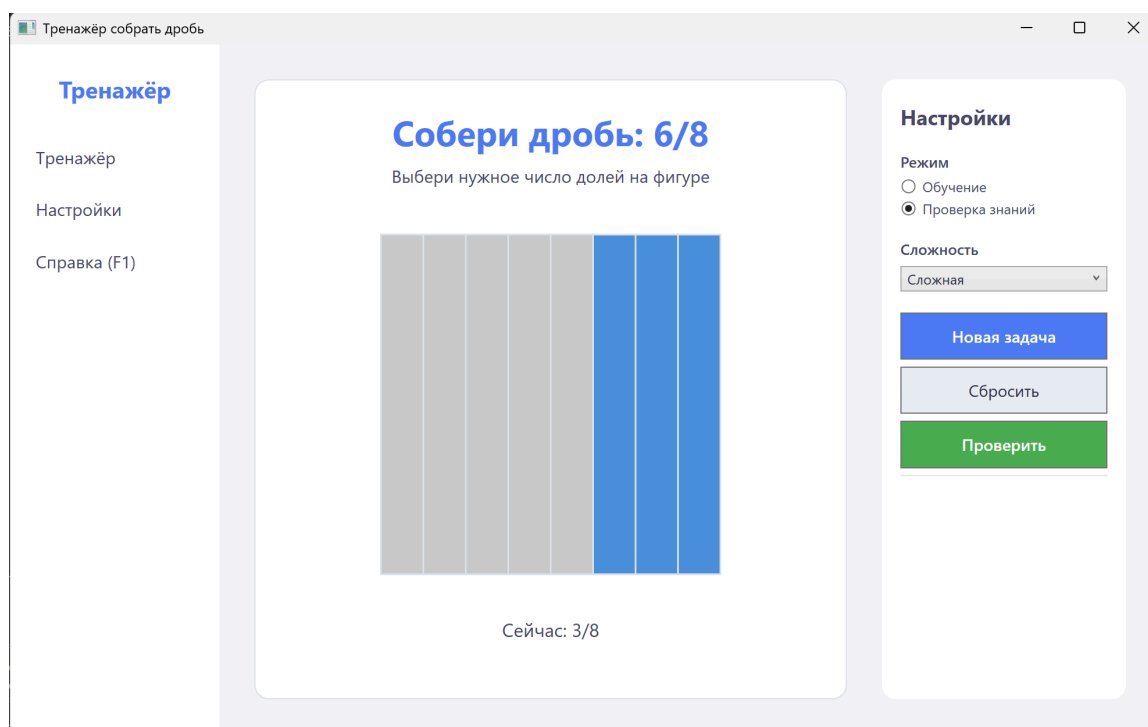


Рисунок 5 – Режим сбора дроби из секторов фигуры

4.2 Определение дроби по фигуре

При выборе типа «Какая дробь показана» фигура отображается с предварительно выделенными секторами. Пользователь должен ввести числитель и знаменатель в текстовые поля. Ответ проверяется на эквивалентность целевой дроби. На рисунке 6 представлена фигура круга с выделенными долями и поля ввода для ответа.

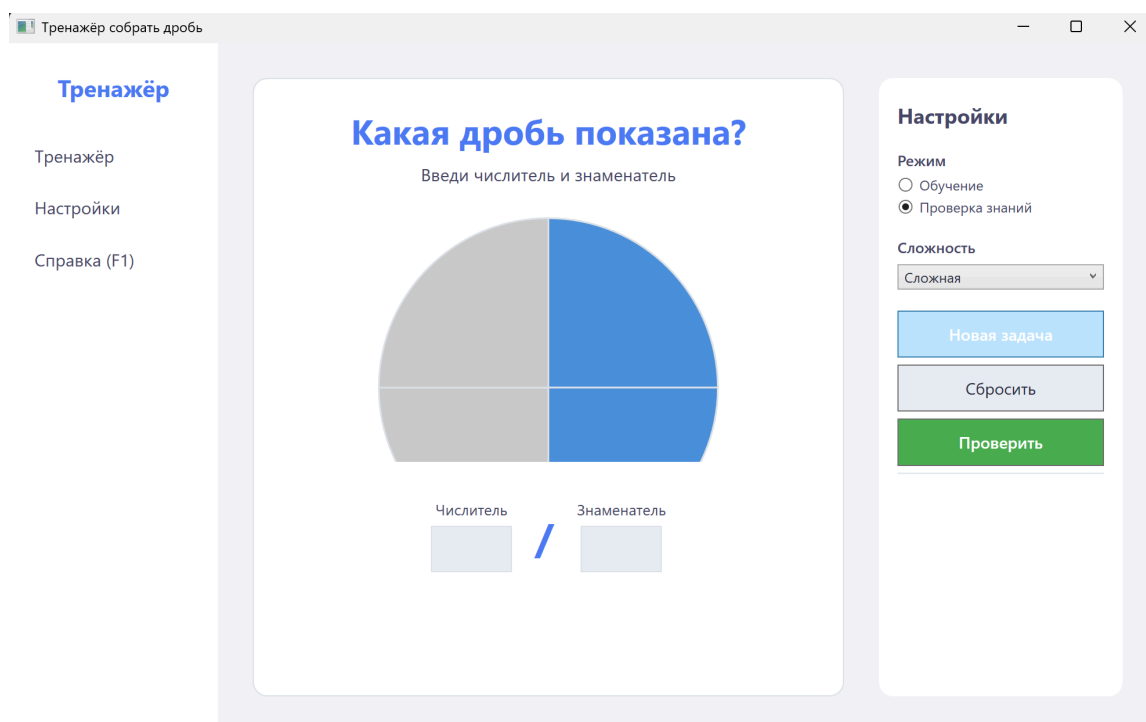


Рисунок 6 – Определение дроби по визуальному представлению

4.3 Сравнение дробей

В режиме «Сравнить дроби» пользователю предлагаются две дроби. Нужно выбрать, какая из них больше. Фигура при этом не отображается — задание ориентировано на понимание соотношения дробей.

4.4 Дополнение до целого

При задании «Дополни до целого» пользователь видит фигуру с частью выделенных секторов и должен указать, сколько долей не хватает до целого. Ответом является числитель недостающей дроби. На рисунке 7 показан круг с тремя выделенными секторами из четырёх — пользователь должен ответить, что не хватает одной доли.

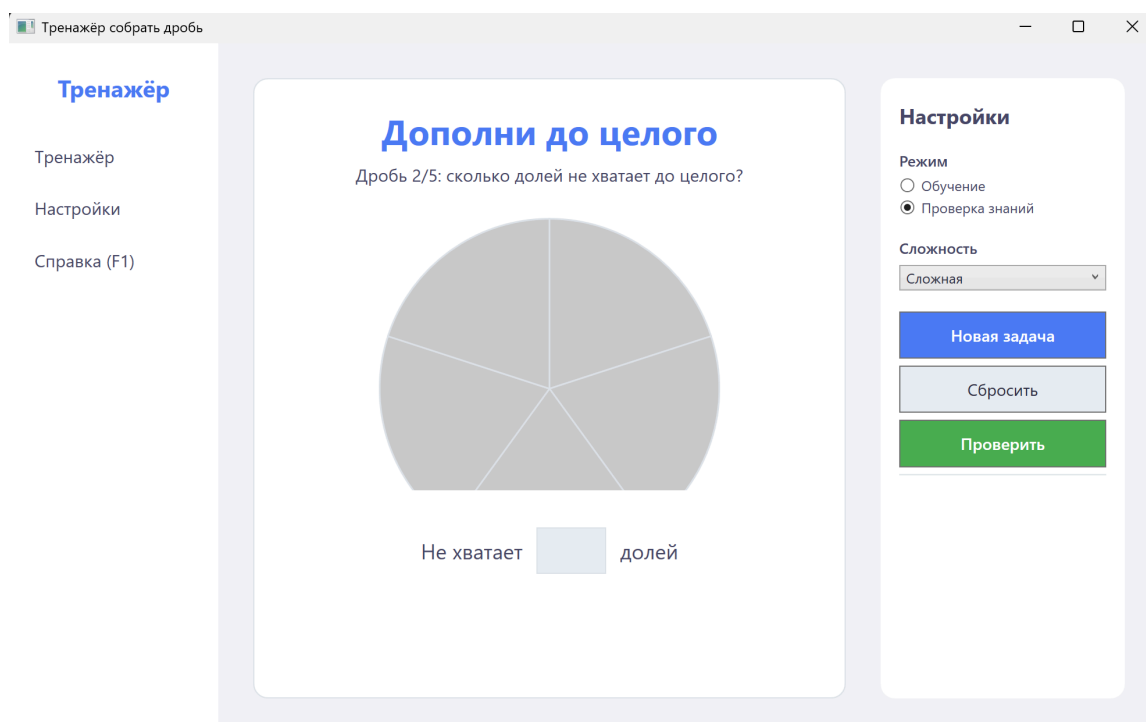


Рисунок 7 – Дополнение дроби до целого

4.5 Поиск пар эквивалентных дробей

Режим «Найти пары» представляет собой карточную игру с шестью карточками. На каждой карточке изображена геометрическая фигура с определённой дробью. Пользователь должен находить пары карточек с эквивалентными дробями.

4.6 Использование справки

Справка вызывается через пункт боковой панели или клавишу F1. В ней перечислены горячие клавиши и основные действия приложения. На рисунке 8 показано окно справочной информации с описанием всех доступных действий.

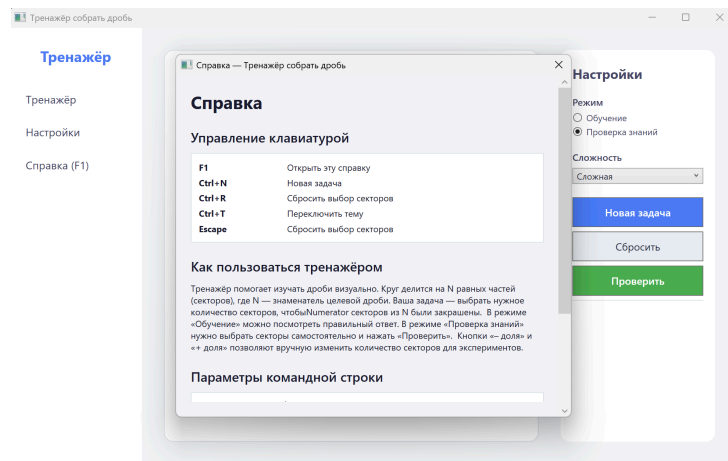


Рисунок 8 – Окно справочной информации

Заключение

В ходе выполнения курсового проекта разработано приложение FractionTrainer, предназначенное для интерактивного изучения дробей. Реализована модель дроби с поддержкой сокращения и проверки эквивалентности, а также модель головоломки с управлением набором секторов.

Разработан кастомный WPF-компонент, отрисовывающий восемь типов геометрических фигур (круг, прямоугольник, треугольник, пятиугольник, шестиугольник, ромб, крест, звезда) и разбивающий их на равные секторы. Реализованы шесть типов заданий: сбор дроби, определение дроби, сравнение дробей, дополнение до целого, поиск эквивалентной дроби и поиск пар.

Приложение поддерживает два режима работы (обучение и проверку), три уровня сложности, светлую и тёмную темы оформления, настройку акцентного цвета. Реализованы хранение настроек и результатов в формате JSON, горячие клавиши, консольная версия и автоматизированные тесты.

Разделение решения на проекты FractionTrainer.Core, FractionTrainer.WPF, FractionTrainer.Console и FractionTrainer.Tests позволило отделить бизнес-логику от пользовательского интерфейса. Использование интерфейсов, наследования, событий и паттерна MVVM обеспечивает расширяемость программы и соответствует принципам объектно-ориентированного проектирования.

Разработанное приложение может быть расширено следующими возможностями: дополнительные типы геометрических фигур, история решённых задач, рейтинговая система, таймер для режима проверки, сохранение прогресса обучения, экспорт результатов в CSV и интеграция с образовательными платформами.

Список литературы

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
2. Microsoft. Windows Presentation Foundation documentation [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/desktop/wpf/> (дата обращения: 21.06.2026).
3. Microsoft. C# documentation [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/csharp/> (дата обращения: 21.06.2026).
4. Microsoft. MVVM documentation [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/architecture/modern-web-apps-azure/architectural-principles#mvvm> (дата обращения: 21.06.2026).
5. Microsoft. Serialize and deserialize JSON using C# [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/overview> (дата обращения: 21.06.2026).
6. Microsoft. Dependency properties overview [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/desktop/wpf/advanced/dependency-properties-overview> (дата обращения: 21.06.2026).
7. Microsoft. Data binding overview [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/desktop/wpf/data/data-binding-overview> (дата обращения: 21.06.2026).
8. xUnit. Official documentation [Электронный ресурс]. URL: <https://xunit.net/docs/> (дата обращения: 21.06.2026).